

Admin Panel — Project Setup & Architecture

1. Prerequisites & Local Setup

```
node >= 18.17.0
npm >= 9.x

# Clone and install
git clone <repo>
cd admin-panel
npm install

# Copy and fill environment variables
cp .env.example .env.local

# Start dev server
npm run dev          # http://localhost:3000
npm run build        # production build
npm run lint         # ESLint check
```

2. Environment Variables

```
# .env.local

# Django backend base URL (no trailing slash)
NEXT_PUBLIC_API_BASE_URL=http://localhost:8000/api/v1

# WebSocket server URL
NEXT_PUBLIC_WS_URL=ws://localhost:8000

# next-auth (must match exactly, including protocol)
NEXTAUTH_URL=http://localhost:3000
NEXTAUTH_SECRET=your-super-secret-key-min-32-chars

# Optional: Google Maps embed key for booking maps
NEXT_PUBLIC_GOOGLE_MAPS_KEY=AIza...
```

Variable	Required	Description
NEXT_PUBLIC_API_BASE_URL	Yes	Django REST API base URL
NEXT_PUBLIC_WS_URL	Yes	WebSocket server base URL
NEXTAUTH_URL	Yes	Canonical URL of the Next.js app
NEXTAUTH_SECRET	Yes	JWT signing secret (min 32 chars)
NEXT_PUBLIC_GOOGLE_MAPS_KEY	No	For booking tracking maps

3. NextAuth Configuration

lib/auth.ts

```
import type { NextAuthOptions } from "next-auth";
import CredentialsProvider from "next-auth/providers/credentials";
import axios from "axios";

export const authOptions: NextAuthOptions = {
  providers: [
    CredentialsProvider({
      name: "Admin Credentials",
      credentials: {
        email: { label: "Email", type: "email" },
        password: { label: "Password", type: "password" },
      },
      async authorize(credentials) {
        if (!credentials?.email || !credentials?.password) return null;

        try {
          const res = await axios.post(
            `${process.env.NEXT_PUBLIC_API_BASE_URL}/admin/auth/login/`,
            {
              email: credentials.email,
              password: credentials.password,
            },
            { headers: { "Content-Type": "application/json" } }
          );

          const data = res.data;

          if (data?.access && data?.user) {
            return {
              id: String(data.user.id),
              name: data.user.name,
```

```

        email: data.user.email,
        role: data.user.role,
        accessToken: data.access,
        refreshToken: data.refresh,
    };
    }
    return null;
} catch {
    return null;
}
},
}),
],

session: {
    strategy: "jwt",
    maxAge: 8 * 60 * 60, // 8 hours
},

jwt: {
    maxAge: 8 * 60 * 60,
},

callbacks: {
    async jwt({ token, user }) {
        if (user) {
            token.accessToken = (user as any).accessToken;
            token.refreshToken = (user as any).refreshToken;
            token.role = (user as any).role;
            token.id = user.id;
        }
        return token;
    },

    async session({ session, token }) {
        session.user.id = token.id as string;
        session.user.role = token.role as string;
        (session as any).accessToken = token.accessToken;
        return session;
    },
},

pages: {
    signIn: "/login",
    error: "/login",
},

debug: process.env.NODE_ENV === "development",
};

```

app/api/auth/[...nextauth]/route.ts

```
import NextAuth from "next-auth";
import { authOptions } from "@lib/auth";

const handler = NextAuth(authOptions);
export { handler as GET, handler as POST };
```

TypeScript augmentation — types/next-auth.d.ts

```
import "next-auth";
import "next-auth/jwt";

declare module "next-auth" {
  interface User {
    id: string;
    role: string;
    accessToken: string;
    refreshToken: string;
  }

  interface Session {
    user: {
      id: string;
      name?: string | null;
      email?: string | null;
      role: string;
    };
    accessToken: string;
  }
}

declare module "next-auth/jwt" {
  interface JWT {
    id: string;
    role: string;
    accessToken: string;
    refreshToken: string;
  }
}
```

4. Middleware — Route Protection

```
// middleware.ts
import { withAuth } from "next-auth/middleware";
import { NextResponse } from "next/server";

export default withAuth(
  function middleware(req) {
```

```

    const token = req.nextauth.token;

    // Optional: restrict to admin role only
    if (token?.role !== "admin") {
        return NextResponse.redirect(new URL("/login?
            error=unauthorized", req.url));
    }

    return NextResponse.next();
},
{
    callbacks: {
        authorized: ({ token }) => !!token,
    },
}
);

export const config = {
    matcher: [
        "/(dashboard)/:path*",
        "/guards/:path*",
        "/users/:path*",
        "/bookings/:path*",
        "/payments/:path*",
        "/sos/:path*",
        "/analytics/:path*",
        "/verifications/:path*",
        "/settings/:path*",
        // Catch-all for App Router group routes
        "/((?!login|api|_next/static|_next/image|favicon.ico).*)",
    ],
};

```

5. Root Layout & Providers

app/layout.tsx

```

import type { Metadata } from "next";
import { Inter } from "next/font/google";
import "./globals.css";
import { Providers } from "@components/Providers";

const inter = Inter({ subsets: ["latin"] });

export const metadata: Metadata = {
    title: "b-secure Admin",
    description: "Internal operations dashboard",
    robots: "noindex, nofollow",
};

```

```

export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang="en" suppressHydrationWarning>
      <body className={inter.className}>
        <Providers>{children}</Providers>
      </body>
    </html>
  );
}

```

components/Providers.tsx

```

"use client";

import { SessionProvider } from "next-auth/react";
import { QueryClientProvider } from "@tanstack/react-query";
import { ReactQueryDevtools } from "@tanstack/react-query-devtools";
import { queryClient } from "@lib/queryClient";

export function Providers({ children }: { children:
React.ReactNode }) {
  return (
    <SessionProvider>
      <QueryClientProvider client={queryClient}>
        {children}
        {process.env.NODE_ENV === "development" && (
          <ReactQueryDevtools initialIsOpen={false} />
        )}
      </QueryClientProvider>
    </SessionProvider>
  );
}

```

6. Dashboard Layout

app/(dashboard)/layout.tsx

```

import { Sidebar } from "@components/layout/Sidebar";
import { Topbar } from "@components/layout/Topbar";

export default function DashboardLayout({
  children,
}: {
  children: React.ReactNode;

```

```

}) {
  return (
    <div className="flex h-screen overflow-hidden bg-gray-50">
      <Sidebar />
      <div className="flex flex-1 flex-col overflow-hidden">
        <Topbar />
        <main className="flex-1 overflow-y-auto p-6">{children}</
main>
      </div>
    </div>
  );
}

```

7. Sidebar Component

```

// components/layout/Sidebar.tsx
"use client";

import Link from "next/link";
import { usePathname } from "next/navigation";
import { cn } from "@lib/utils";
import { useSosStore } from "@stores/sosStore";
import { useVerificationCount } from "@hooks/useVerificationCount";
import {
  LayoutDashboard,
  ShieldCheck,
  Users,
  CalendarCheck,
  CreditCard,
  AlertTriangle,
  BarChart3,
  BadgeCheck,
  Settings,
} from "lucide-react";
import { Badge } from "@components/ui/badge";

interface NavItem {
  href: string;
  label: string;
  icon: React.ComponentType<{ className?: string }>;
  badge?: React.ReactNode;
}

export function Sidebar() {
  const pathname = usePathname();
  const { activeEvents } = useSosStore();
  const { pendingCount } = useVerificationCount();

  const navItems: NavItem[] = [
    {

```

```

      href: "/",
      label: "Dashboard",
      icon: LayoutDashboard,
    },
    {
      href: "/guards",
      label: "Guards",
      icon: ShieldCheck,
    },
    {
      href: "/users",
      label: "Users",
      icon: Users,
    },
    {
      href: "/bookings",
      label: "Bookings",
      icon: CalendarCheck,
    },
    {
      href: "/payments",
      label: "Payments",
      icon: CreditCard,
    },
    {
      href: "/sos",
      label: "SOS",
      icon: AlertTriangle,
      badge:
        activeEvents.length > 0 ? (
          <Badge className="bg-red-500 text-white animate-pulse
text-xs px-1.5 py-0.5">
            {activeEvents.length}
          </Badge>
        ) : null,
    },
    {
      href: "/analytics",
      label: "Analytics",
      icon: BarChart3,
    },
    {
      href: "/verifications",
      label: "Verifications",
      icon: BadgeCheck,
      badge:
        pendingCount > 0 ? (
          <Badge variant="secondary" className="text-xs px-1.5
py-0.5">
            {pendingCount}
          </Badge>
        ) : null,
    },
    {

```

```

        href: "/settings",
        label: "Settings",
        icon: Settings,
    },
];

return (
    <aside className="w-64 shrink-0 border-r border-gray-200 bg-white flex flex-col">
        {/* Logo */}
        <div className="flex h-16 items-center border-b border-gray-200 px-6">
            <span className="text-xl font-bold text-gray-900">b-secure</span>
            <span className="ml-2 text-xs font-medium text-gray-400 uppercase tracking-wide">Admin</span>
        </div>

        {/* Navigation */}
        <nav className="flex-1 overflow-y-auto px-3 py-4 space-y-1">
            {navItems.map((item) => {
                const isActive =
                    item.href === "/"
                    ? pathname === "/"
                    : pathname.startsWith(item.href);

                return (
                    <Link
                        key={item.href}
                        href={item.href}
                        className={cn(
                            "flex items-center justify-between rounded-lg px-3 py-2.5 text-sm font-medium transition-colors",
                            isActive
                                ? "bg-gray-900 text-white"
                                : "text-gray-600 hover:bg-gray-100 hover:text-gray-900"
                        )}
                    >
                        <div className="flex items-center gap-3">
                            <item.icon className="h-4 w-4 shrink-0" />
                            {item.label}
                        </div>
                        {item.badge}
                    </Link>
                );
            })}
        </nav>

        {/* Footer */}
        <div className="border-t border-gray-200 p-4">
            <p className="text-xs text-gray-400 text-center">

```

```

        b-secure © {new Date().getFullYear()}
      </p>
    </div>
  </aside>
);
}

```

8. Topbar Component

```

// components/layout/Topbar.tsx
"use client";

import { usePathname } from "next/navigation";
import { signOut, useSession } from "next-auth/react";
import { Avatar, AvatarFallback } from "@components/ui/avatar";
import {
  DropdownMenu,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuSeparator,
  DropdownMenuTrigger,
} from "@components/ui/dropdown-menu";
import { useWsStore } from "@stores/wsStore";
import { Button } from "@components/ui/button";
import { LogOut, RefreshCw, Wifi, WifiOff } from "lucide-react";
import { cn } from "@lib/utils";

function useBreadcrumb(pathname: string): string[] {
  const segments = pathname.split("/").filter(Boolean);
  return segments.map((s) =>
    s.charAt(0).toUpperCase() + s.slice(1).replace(/-/g, " ")
  );
}

export function Topbar() {
  const pathname = usePathname();
  const { data: session } = useSession();
  const { connected, reconnect } = useWsStore();
  const crumbs = useBreadcrumb(pathname);

  const initials = session?.user?.name
    ?.split(" ")
    .map((n) => n[0])
    .join("")
    .toUpperCase()
    .slice(0, 2) ?? "AD";

  return (
    <header className="flex h-16 shrink-0 items-center justify-
between border-b border-gray-200 bg-white px-6">
      { /* Breadcrumb */ }
    </header>
  );
}

```

```

<nav className="flex items-center gap-2 text-sm">
  <span className="text-gray-400">Admin</span>
  {crumbs.map((crumb, i) => (
    <span key={i} className="flex items-center gap-2">
      <span className="text-gray-300">/</span>
      <span
        className={cn(
          i === crumbs.length - 1
            ? "font-semibold text-gray-900"
            : "text-gray-500"
        )}
      >
        {crumb}
      </span>
    </span>
  ))}
</nav>

{/* Right section */}
<div className="flex items-center gap-4">
  {/* WebSocket status dot */}
  <div className="flex items-center gap-2">
    {connected ? (
      <div className="flex items-center gap-1.5">
        <Wifi className="h-4 w-4 text-green-500" />
        <span className="text-xs text-green-600 font-
medium">Live</span>
      </div>
    ) : (
      <div className="flex items-center gap-1.5">
        <WifiOff className="h-4 w-4 text-red-500" />
        <Button
          variant="ghost"
          size="sm"
          className="h-6 px-2 text-xs text-red-600"
          onClick={reconnect}
        >
          <RefreshCw className="h-3 w-3 mr-1" />
          Reconnect
        </Button>
      </div>
    )}
  </div>

  {/* Avatar dropdown */}
  <DropdownMenu>
    <DropdownMenuTrigger asChild>
      <button className="flex items-center gap-2 rounded-
full outline-none focus:ring-2 focus:ring-gray-300">
        <Avatar className="h-8 w-8">
          <AvatarFallback className="bg-gray-900 text-white
text-xs">
            {initials}
          </AvatarFallback>

```

```

        </Avatar>
      </button>
    </DropdownMenuTrigger>
    <DropdownMenuContent align="end" className="w-52">
      <div className="px-3 py-2">
        <p className="text-sm font-medium">{session?.user?.name}</p>
        <p className="text-xs text-gray-500">{session?.user?.email}</p>
      </div>
      <DropdownMenuSeparator />
      <DropdownMenuItem
        className="text-red-600 focus:text-red-600 cursor-pointer"
        onClick={() => signOut({ callbackUrl: "/login" })}
      >
        <LogOut className="mr-2 h-4 w-4" />
        Sign out
      </DropdownMenuItem>
    </DropdownMenuContent>
  </DropdownMenu>
</div>
</header>
);
}

```

9. Axios Instance

```

// services/axios.ts
import axios, { AxiosError, InternalAxiosRequestConfig } from
  "axios";
import { getSession } from "next-auth/react";

export const apiClient = axios.create({
  baseURL: process.env.NEXT_PUBLIC_API_BASE_URL,
  headers: {
    "Content-Type": "application/json",
  },
  timeout: 15_000,
});

// Inject auth token on every request
apiClient.interceptors.request.use(
  async (config: InternalAxiosRequestConfig) => {
    const session = await getSession();
    if (session?.accessToken) {
      config.headers.Authorization = `Bearer ${
        session.accessToken
      }`;
    }
    return config;
  },

```

```

    (error) => Promise.reject(error)
  );

  // Global error handling
  apiClient.interceptors.response.use(
    (response) => response,
    async (error: AxiosError) => {
      if (error.response?.status === 401) {
        // Token expired – redirect to login
        if (typeof window !== "undefined") {
          window.location.href = "/login?error=session_expired";
        }
      }

      const message =
        (error.response?.data as any)?.detail ||
        (error.response?.data as any)?.message ||
        error.message ||
        "An unexpected error occurred";

      return Promise.reject(new Error(message));
    }
  );

```

10. React Query Client

```

// lib/queryClient.ts
import { QueryClient } from "@tanstack/react-query";

export const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 30_000, // 30 seconds
      gcTime: 5 * 60_000, // 5 minutes
      retry: 1,
      refetchOnWindowFocus: false,
    },
    mutations: {
      onError: (error: unknown) => {
        console.error("[Mutation Error]", error);
      },
    },
  },
});

```

11. TypeScript Path Aliases

tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2017",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "preserve",
    "incremental": true,
    "plugins": [{ "name": "next" }],
    "paths": {
      "@/*": ["./*"]
    }
  },
  "include": ["next-env.d.ts", "**/*.ts", "**/*.tsx", ".next/
    types/**/*.ts"],
  "exclude": ["node_modules"]
}
```

12. ESLint Config

```
// .eslintrc.json
{
  "extends": ["next/core-web-vitals", "next/typescript"],
  "rules": {
    "@typescript-eslint/no-explicit-any": "warn",
    "@typescript-eslint/no-unused-vars": ["error", {
      "argsIgnorePattern": "^_" }],
    "react-hooks/exhaustive-deps": "warn"
  }
}
```

13. Utility Helpers

```
// lib/utils.ts
import { type ClassValue, clsx } from "clsx";
```

```

import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

export function formatCurrency(amount: number, currency = "ZAR"):
  string {
  return new Intl.NumberFormat("en-ZA", {
    style: "currency",
    currency,
    minimumFractionDigits: 2,
  }).format(amount);
}

export function formatDate(
  dateString: string,
  opts?: Intl.DateTimeFormatOptions
): string {
  return new Intl.DateTimeFormat("en-ZA", {
    day: "2-digit",
    month: "short",
    year: "numeric",
    ...opts,
  }).format(new Date(dateString));
}

export function formatDateTime(dateString: string): string {
  return formatDate(dateString, {
    day: "2-digit",
    month: "short",
    year: "numeric",
    hour: "2-digit",
    minute: "2-digit",
  });
}

export function elapsedTime(startIso: string): string {
  const diff = Math.floor((Date.now() - new
    Date(startIso).getTime()) / 1000);
  const h = Math.floor(diff / 3600);
  const m = Math.floor((diff % 3600) / 60);
  const s = diff % 60;
  if (h > 0) return `${h}h ${m}m`;
  if (m > 0) return `${m}m ${s}s`;
  return `${s}s`;
}

```